Drift-Resilient Time Series Forecasting

2625170 Wai Lee Boo

***Abstract*—Time series forecasting performance often degrades under concept drift, where the underlying data distribution changes over time. This project proposes Trial-Based Drift-Triggered Retraining (TDTR), a framework that monitors prediction error using statistical drift detectors and retrains a challenger model only when drift is detected, adopting it only if it outperforms the current model over a fixed evaluation horizon. TDTR is evaluated across neural architectures (LSTM, ELM, PSO-LSTM, PSO-ELM) and conventional machine learning models (Random Forest, SVR), on synthetic drift benchmarks and real S&P 500 data, along with an ablation study using continuous retraining. On real-world data, TDTR reduces Price_MAE by 41% for LSTM and 31% for PSO-LSTM, while ELM-based models show no benefit due to strong static generalisation. On synthetic recurring benchmarks, results are mixed and drift-type dependent, with PSO-ELM performing best overall. The ablation confirms that fixed-interval retraining matches TDTR in accuracy, but at roughly double the computational cost. Under genuinely unseen concepts with limited training coverage, TDTR produces its largest gains of up to 70%. Overall, the benefit of TDTR depends on how well the static model generalises to the incoming concept distribution.**

***Index Terms*—Machine Learning, Time series forecasting, Concept Drift, Drift Detector, Adaptive forecasting, Long Short Term Memory (LSTM)**

I. Introduction

Time series forecasting is one of the core tasks in machine learning, with applications ranging from sensor monitoring, energy demand usage, healthcare signals, and the financial market [1]–[3]. In these industries, high forecast precision allows earlier interventions, reduces operational costs, and identifies profit opportunities. However, real-world deployment faces a significant challenge: data is often non-stationary, meaning the statistical relationship between past observations and future outcomes evolves over time due to seasonal shifts, environmental changes, or external shocks [4]. In streaming settings, these changes are called concept drift [5], where the underlying data-generating distribution varies, invalidating the assumption that training and test data are identically distributed [6]. Consequently, models trained offline often suffer from progressive performance degradation after deployment, even if they performed well on historical benchmarks [7].

Existing drift-aware forecasting systems struggle to balance responsiveness with stability. Most detectors rely on strict thresholds, and aggressive settings catch drift early but trigger frequent false alarms, leading to unnecessary updates that destabilise the model [8]. Conversely, conservative settings can completely miss critical market shifts [9]. Although some recent methods manage these shifts using ensembles or dynamic swarms, they typically do so by maintaining multiple active models simultaneously. This creates a massive computational overhead. Consequently, there remains a clear gap for a lightweight, single model framework, one that explicitly detects drifts, but strictly validates any proposed update before actually deploying it.

This project investigates drift-adaptive forecasting in both controlled synthetic drift scenarios and real-world financial data. It compares LSTM and Extreme Learning Machine models (ELM, a single hidden-layer neural network), along with their variants based on Particle Swarm Optimisation (PSO).

The proposed framework, **Trial-Based Drift-Triggered Retraining** (TDTR), monitors prediction error over time using statistical drift detection and retrains the model when changes in the data are detected. A retrained "challenger" model is only adopted if it performs better than the current model over a short evaluation period, helping balance adaptation with stability. In addition to the standard drift setting, this work also explores more realistic scenarios by answering the following research questions:

- **RQ1:** To what extent does PSO-based optimisation of output layer weights improve the robustness of neural forecasting models (ELM and LSTM) under different types of concept drift? By looking at recurring concept patterns, where similar behaviours reappear over time, this assesses whether models can reuse previously learned patterns.
- **RQ2:** How effectively does Trial-Based Drift-Triggered Retraining (TDTR) improve forecasting performance compared to non-adaptive baselines across synthetic drift scenarios and real financial data? This establishes the core evaluation of TDTR, assessing if it is still necessary in settings where patterns reappear over time.
- **RQ3:** How does continuous online retraining without a drift detector compare to TDTR under identical retraining windows and computational constraints across different drift types? This contributes to evaluating whether adaptive retraining architectures are necessary in these recurring settings.
- **RQ4:** Can conventional machine learning methods such as Random Forest and Support Vector Regression (SVR) benefit from drift adaptation within the TDTR framework, and do they exhibit the same ceiling effect observed in analytical neural models? This also looks at recurring concept patterns, extending the core TDTR evaluation of RQ2 by examining its behaviour across additional model classes.
- **RQ5:** Does the TDTR framework improve forecasting performance over non-adaptive baselines when models are trained on limited concept coverage and evaluated on unseen concept configurations? This considers a limited training setting, where models are trained on a small portion of data and then tested on a mixture of new and unseen concepts, reflecting situations where future data may differ from what was previously observed. It extends the core TDTR evaluation of RQ2 under more challenging unseen concept configurations.

This paper makes several contributions to adaptive machine learning for financial time series forecasting. First, it presents

a complete implementation and evaluation of the TDTR framework. Second, it provides a comprehensive empirical evaluation on both synthetic and real S&P 500 data. Third, an ablation study is conducted to isolate the effect of drift detection from periodic retraining. Fourth, the framework is extended to conventional machine learning methods, including Random Forest and SVR, to examine whether the findings generalise beyond neural models. Finally, a training coverage analysis investigates whether the benefits of adaptation in analytical models are driven by data availability rather than architectural limitations.

The remainder of this paper is structured as follows: Section II presents the problem definition. Section III reviews related work. Section IV describes the proposed system architecture and methodological framework. Sections V and VI outline the experimental design and present the empirical results addressing RQ$1-5$, and Section VII concludes the paper and provides directions for future research.

II. Problem Definition

Consider a streaming time series of observations where y_t represents the target variable (e.g., the daily return of a stock price) at time t. Unlike standard supervised learning where features and targets represent distinct attributes, time series models rely on historical values of the target itself. This data was structured using a sliding window approach. Specifically, for a given window of size w, the input vector X_t consists of the w most recent observations: $X_t = \{y_{t-w+1}, \ldots, y_{t-1}, y_t\}$

After observing X_t, a forecasting model f_{θ_t} produces a one step ahead prediction for the next day, $\hat{y}_{t+1} = f_{\theta_t}(X_t)$. In standard supervised learning, the model is trained on historical examples to approximate $P(y_{t+1}|X_t)$ under the assumption that the joint distribution $P(X_t, y_{t+1})$ is stationary. However, under concept drift, this assumption is violated because the joint probability distribution changes between time t and $t+\Delta$ for some $\Delta > 0$:

$$P_t(X, y) \neq P_{t+\Delta}(X, y)$$

When this occurs, a model trained on older data can become increasingly misaligned with the current regime, leading to gradual or abrupt drops in predictive performance. This degradation is especially problematic in high-noise and rapidly changing domains such as financial markets, where shifts in volatility and market sentiment can alter the input-output relationship. As a result, forecasting systems must not only achieve low error during stable periods but also remain reliable when regimes change either by detecting distribution shifts early, adapting the model efficiently, or maintaining robustness to avoid overreacting to noise. The central problem addressed in this paper is therefore, how to preserve next-day forecasting accuracy under evolving distributions while managing the trade-off between fast adaptation and stable behaviour. In particular, this work investigates whether drift-aware retraining strategies can improve robustness compared to static and continuously retrained baselines under different drift scenarios.

III. Related Work

Recent studies have increasingly focused on drift-aware evaluation frameworks and adaptive learning methods for non-stationary environments. As a result, a wide range of approaches have been developed to identify distributional shifts, update models efficiently, and preserve strong performance across changing regimes. These include methods based on explicit drift detection and retraining, as well as online ensembling and evolutionary adaptation. The following subsections outline three key approaches from related work that are relevant to this paper.

A. Offline LSTM forecasting

Numerous studies have explored LSTM networks for stock market prediction under conventional offline training settings. For example, Nelson et al. [10] and Krauss [11] apply LSTM models to stock market forecasting and show that deep recurrent architectures can outperform traditional machine learning baselines in financial time-series forecasting. However, as with many LSTM-based approaches, including Bao et al. [12], these models are trained using a standard batch setup with a fixed training and testing split. Although such studies highlight the ability of LSTM models to capture temporal dependencies in financial data, they typically assume a stable relationship between past and future observations and do not explicitly address concept drift, online adaptation, or dynamic retraining. In contrast, this paper retains LSTM as a baseline forecasting model while evaluating it under drift-aware protocols and incorporating detector-triggered retraining and trial-based adaptation to better handle evolving market regimes. It further investigates online adaptation without an explicit drift detector by enabling continuous learning over time.

B. Ensemble-Based Approaches for Concept Drift

Some studies have also explored ensemble-based approaches to address concept drift in time-series forecasting. Zhang et al. propose OneNet, an online ensembling framework that combines two forecasting models and dynamically adjusts their weights over time based on recent prediction performance [13]. By continuously re-weighting the contribution of each model, the system can adapt to changing data distributions without explicitly detecting drift. A similar ensemble-based strategy is presented in Mitigating Concept Drift Challenges in Evolving Smart Grids, which proposes an adaptive ensemble LSTM for enhanced load forecasting [1]. This approach takes advantage of the diversity of several LSTM predictors to increase robustness under changing energy demand patterns. While both methods demonstrate how ensemble strategies can mitigate the effects of concept drift in non-stationary environments, they rely on maintaining multiple models simultaneously. In contrast, this paper maintains a single active forecasting model and adapts only after a drift trigger, using trial-based evaluation to determine whether a new model should replace the current one. This drift triggered, trial-based mechanism can reduce computational overhead compared to continuously maintaining multiple models.

C. PSO-driven Forecasting under Concept Drift

Oliveira et al. propose a Swarm Intelligence for Series with Concept Drift (SISC), a framework for time-series forecasting that uses dynamic swarm intelligence to handle evolving

regimes [14]. In SISC, groups of particles explore and refine the parameters of the prediction models over time, allowing the system to adjust as the underlying data distribution changes. The framework also stores previously successful swarm configurations so that they can be reused when similar concepts reappear. At its core, the framework relies on Particle Swarm Optimisation (PSO), which searches for well-performing parameter settings by combining individual particle experience with information shared across the swarm [15]. In drift aware forecasting, PSO is useful because it can quickly re-optimise model parameters after distributional change without relying on back-propagation. Oliveira et al. also show how PSO-based methods can be used to update predictors in drifting time-series settings. This is closely related to this paper, which also uses PSO-based optimisation under drift. However, unlike SISC, which continuously evolves and manages multiple swarms as the data distribution changes, the approaches used in this study apply PSO only after explicit drift detection and use a trial-based challenger mechanism to decide whether a newly retrained candidate should replace the current active forecasting model.

In general, existing approaches to drift-aware forecasting highlight the trade-off between continuous adaptation and computational efficiency. Ensemble and swarm-based methods maintain multiple models simultaneously to handle evolving distributions, typically combining their predictions through weighted averaging or voting mechanisms. Although this improves robustness, it increases computational cost and can be difficult to scale.

This paper takes a different approach by maintaining a single active forecasting model while using a trial-based mechanism to evaluate candidate models only when drift is detected. Rather than averaging predictions together like traditional ensembles, the TDTR framework requires candidate models to compete directly against one another in a strict performance evaluation. Only the single model that proves to be the most accurate during the trial phase is actually deployed. This approach ensures that the system can rapidly adapt to genuine market shifts but avoids the heavy computational burden of keeping multiple active models running simultaneously. Ultimately, this provides a highly efficient balance: the model remains responsive to drift and stable against noise, all while keeping computational costs strictly controlled.

IV. System Overview

This section introduces the proposed **Trial-Based Drift-Triggered Retraining (TDTR)** framework for time-series forecasting under concept drift. Figure 1 summarises TDTR as three blocks, illustrating the transition from offline model initialisation to online prediction and drift detection, followed by trial-based evaluation and selection of challenger models.

Block A initialises *TDTR* by training an initial forecasting model on historical data. Two base architectures are considered — an Extreme Learning Machine (ELM) and an LSTM [16], and PSO-optimised variants, PSO-ELM and PSO-LSTM. Particle Swarm Optimisation (PSO) is a population-based optimiser in which each particle represents a candidate solution vector $x \in \mathbb{R}^D$, where D is the number of optimised variables. Particles update their positions by combining individual experience (personal best) and collective knowledge (global best) [15], [17]. In many traditional PSO-LSTM approaches, each particle encodes the concatenation of all network parameters, including LSTM gate weights and biases and output-layer parameters, which produces a very high-dimensional search space [18]. In contrast, the PSO-LSTM design here restricts the PSO to the final fully connected (FC) output layer only. The recurrent LSTM layers are trained via backpropagation and then frozen, while the PSO optimises the FC mapping from the hidden state to the output. With hidden size $H = 256$, the FC layer contains 256 weights plus a bias term, giving a PSO search dimension of $D = 256 + 1 = 257$. Restricting optimisation to this low-dimensional layer reduces computational cost while still enabling rapid post-drift adjustment of the output mapping.

Block B performs online forecasting on a stream of incoming observations. At each time step, the current active model produces a one-step prediction $\hat{y}_t$, and the absolute error $e_t = |\hat{y}_t - y_t|$ is calculated. TDTR first checks whether a cooldown period is active. During cooldown, drift detection is suppressed to avoid repeated retraining from short-term noise after a recent model update. If the cooldown is inactive, the error e_t is fed into a statistical drift detector. When drift is detected, TDTR extracts the most recent observations from a fixed retraining window and, if sufficient data is available, constructs a challenger by cloning the current model and updating it on this recent window. Retraining is model-dependent. ELM and LSTM are retrained using standard update procedures, while PSO-ELM and PSO-LSTM apply PSO-based re-optimisation during the update stage. For PSO-based retraining, the swarm is initialised using a mixed strategy to balance exploitation and exploration, where 70% of particles retain their previous weights. The worst 15% are reinitialised completely at random, while the next 15% are reinitialised from the current solution with additive Gaussian noise. In practice, this noise is sampled from a normal distribution with small variance, $\mathcal{N}(0, 0.1)$ to ensure that the particles remain close to the current solution while still introducing variability. Larger perturbations would effectively behave like full reinitialisation, reducing the benefit of retaining prior information. This design is motivated by the idea of resetting only the worst particles [14] .

Block C manages adaptation through a bounded challenger pool and a trial-based selection policy. Each time drift-triggered retraining produces a challenger, the challenger is inserted into a pool with a maximum capacity. If the pool is full, the worst performing challenger is removed, based on the rolling mean absolute error over a recent error window, before inserting the new challenger. Importantly, challengers are not adopted immediately. Instead, TDTR enters a fixed-length trial phase of T steps, for example $T = 20$, while both the active model and all challengers accumulate errors in parallel. At the end of the trial, TDTR selects the model with the lowest mean error. If a challenger outperforms the active model, it is promoted to become the new active model, and the displaced active model is retained in the challenger pool with a reset of its error history. If the active model remains best, no switch is performed. This trial-based promotion rule remains responsive

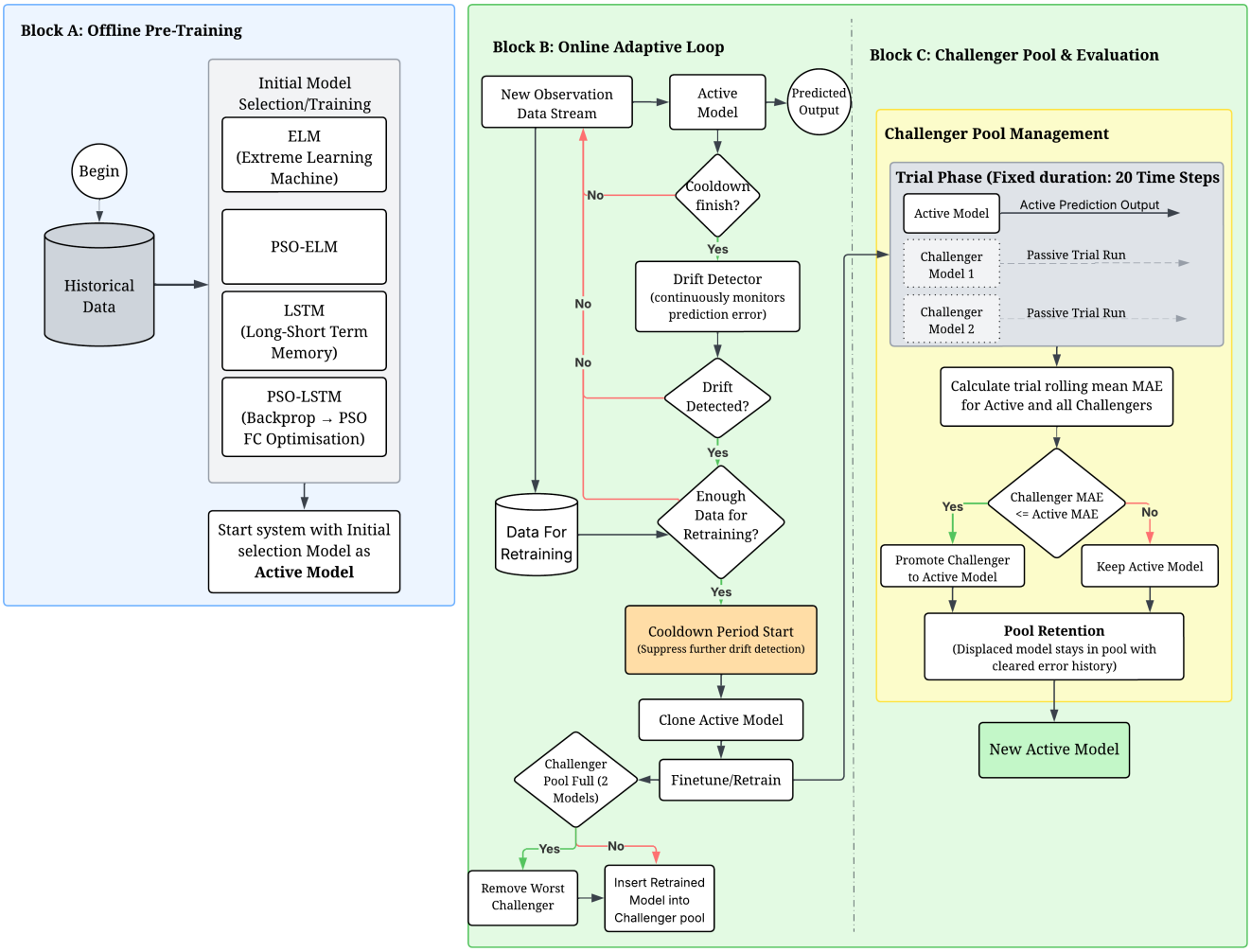

Fig. 1: Trial-Based Drift-Triggered Retraining (TDTR) System Overview

Algorithm 1 Trial-Based Drift-Triggered Retraining (TDTR)

Train initial active model M_{active} on historical data
Initialise challenger pool $\mathcal{P} \leftarrow \emptyset$, buffer $B \leftarrow [\,]$, cooldown $\leftarrow 0$
for each t from w to $|X_{\text{test}}| - 1$ **do**
  Predict $\hat{y}_t$ with M_{active}, compute $e_t = |\hat{y}_t - y_t|$, append y_t to B
  Update rolling errors for M_{active} and all $M \in \mathcal{P}$
  if trial is active and trial steps $= T$ **then**
    Promote the model in $\{M_{\text{active}}\} \cup \mathcal{P}$ with lowest trial MAE
    Reset error histories, set cooldown $\leftarrow C$, end trial
  end if
  if cooldown > 0 **then**
    cooldown $\leftarrow$ cooldown -1; **continue**
  end if
  if detector flags drift on e_t and $|B| \geq R$ **then**
    $M' \leftarrow \textsc{Retrain}(\textsc{DeepCopy}(M_{\text{active}}), B[-R:])$
    Insert M' into $\mathcal{P}$, evict worst if $|\mathcal{P}| > P_{\max}$, start trial
    Reset drift detector
  end if
end for

to drift while being robust to noisy detections, reducing unnecessary model switching. The algorithm 1 summarises the TDTR procedure described in Blocks A–C, showing the online loop, drift-triggered retraining, and the trial-based promotion mechanism.

In addition to detector-triggered TDTR, in **RQ3** a continuous-update variant that completely removes the drift detector was evaluated. Because drift detectors can be difficult to tune, this variant updates the forecasting model online at regular intervals using the most recent data window. This serves as an ablation study that isolates the impact of explicit drift detection and enables a direct comparison between drift-triggered adaptation and a simpler continuous-learning strategy under the same retraining window and computational constraints. A pseudocode description of this detector-free variant is provided in Algorithm 2 in Appendix B.

V. Experimental Setup

This section describes the experimental setup used across all research questions, covering synthetic time series with controlled drift and real financial data. It outlines the parameter settings for RQ1−RQ5, and summarises the statistical tests used to compare models.

A. Synthetic Datasets

Since concept drift boundaries are rarely known in real-world data, drift detection and adaptation are evaluated using synthetic time series with explicitly controlled drift points.

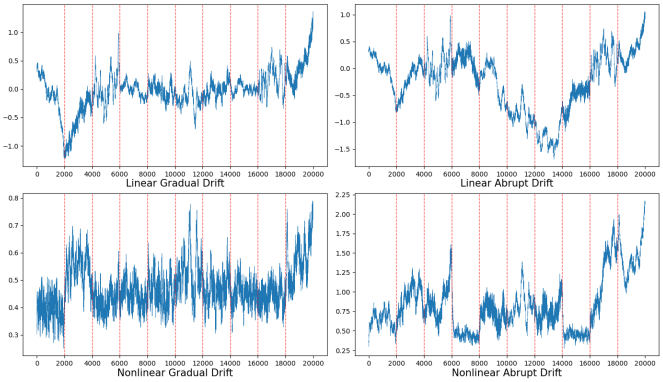

Fig. 2: Representation of each time series with known drifts. Vertical red lines represent known concept drifts

Four synthetic settings are generated by combining two underlying dynamics (linear and non-linear) with two drift types (gradual and abrupt), as shown in Figure 2. To support statistical testing, 30 instances per setting are generated, creating 120 time series in total. The results are reported as the average performance across each group. Each series contains 10 concepts, with 2,000 observations per concept, and drift occurs at concept boundaries. The synthetic series are generated using an autoregressive process, with separate linear and non-linear formulations following Oliveira et al. [14]. From the sixth concept onwards, earlier concepts are repeated, making the series recurring rather than introducing entirely new concepts throughout. The implementation used in this study adapts the original Portuguese generator code to English.

Gradual drift refers to the transition phase in which the probability of samples coming from the old concept decreases over time while the probability of samples from the new concept increases. Abrupt drift occurs when the data generating distribution changes suddenly at time t, so $P_t(x, y)$ is replaced by a different distribution at $t + 1$.

B. Real-World Dataset

A single real-world time series was used: the daily S&P 500 index, containing 18,780 observations collected from May 15, 1950, to December 31, 2024. The forecasting task is to predict the next-day log return of the index using a sliding window of the most recent observations as input features. The index was selected primarily because it offers one of the longest continuous historical daily data available among major financial indices, ensuring the dataset spans multiple distinct market regimes.

C. Drift Detectors

Three statistical drift detection algorithms were initially evaluated on PSO-LSTM, namely *ADWIN* [19], *KSWIN* [20], and *Page-Hinkley* [21], with the best performing detector subsequently used throughout RQ2, RQ4 and RQ5. These three methods were chosen because they represent distinct, well-established paradigms in drift detection: adaptive windowing, distribution based statistical testing, and cumulative error tracking. ADWIN maintains a dynamically sized window of recent observations and splits it into two subwindows. The drift is declared when $|\mu_1 - \mu_2| > \varepsilon$, where the threshold (ε) is derived from Hoeffding's inequality. Page-Hinkley monitors the running mean of incoming observations and accumulates their deviations using a cumulative sum (CUSUM) approach. A drift is detected when the accumulated deviation exceeds a predefined threshold. KSWIN (Kolmogorov-Smirnov Windowing) detects distributional changes by maintaining a fixed-size sliding window and applying the two-sample Kolmogorov–Smirnov test to compare recent and past observations. Drift is declared when the empirical cumulative distributions differ significantly.

D. Friedman and Nemenyi Tests

To compare multiple forecasting models across multiple datasets, the non-parametric Friedman test is used [22]. This test evaluates whether there are statistically significant differences in the average ranks of the models without assuming normality of errors. If the null hypothesis of equal performance is rejected, the Nemenyi post-hoc test is applied to perform pairwise comparisons. The Critical Difference (CD) diagram is used to visualise groups of models that are not significantly different at $\alpha = 0.05$.

E. Model Evaluation

Model performance is evaluated differently for synthetic and real-world data due to the availability of absolute price levels in the financial series. For synthetic datasets, evaluation is performed directly on the series values, using MAE computed between predicted and true values over the test region. For the real-world dataset, the models operate on daily logarithmic returns that are Min-Max scaled, and performance is reported using Return_MAE. Price_MAE is also reported by inverse scaling predicted returns, reconstructing prices from the previous day's price, and comparing predicted versus true market prices in index points.

F. RQ1: Setup

RQ1 compares five offline forecasting models: ARIMA (statistical standard), LSTM (deep sequence learning), ELM (highly efficient analytical mapping), PSO-LSTM, and PSO-ELM. All models are trained offline with no drift detection or online adaptation, using a sliding input window of $w = 10$ for learning-based models. The LSTM uses a single hidden layer with $H = 256$ units and is trained with early stopping. The ELM uses 10 hidden neurons, with output weights solved analytically via the Moore-Penrose pseudoinverse. Due to numerical instability observed during the experiments, this was later replaced with L2-regularised least-squares (ridge regression with $\lambda = 10^{-4}$) as discussed in Section VI-B. PSO-ELM applies PSO to optimise the ELM input weights before the analytical solve. Both PSO-based models use a swarm of 30 particles with up to $1,000$ iterations, and terminate early if no improvement is observed for a fixed patience period. ARIMA is included as a statistical reference baseline.

Data is split chronologically. Synthetic datasets use a 50%/10%/40% split for training, validation, and testing. Out of the 10 concepts per series, this allocates 5 concepts to training, 1 to validation, and 4 to testing. Because earlier patterns repeat from the sixth concept onwards, the synthetic

test region focuses on evaluating performance under *recurring* concepts instead of entirely novel data points. However, the real-world S&P 500 series uses a 70%/10%/20% split.

G. RQ2: Setup

RQ2 evaluates the TDTR framework across two phases. Phase 1 tests three drift detectors (ADWIN, Page-Hinkley, and KSWIN) using the PSO-LSTM model on synthetic data to find the most reliable detector. Phase 2 then applies the winning detector to all four of the adaptive models (LSTM, ELM, PSO-LSTM, and PSO-ELM). To align with real-world financial conditions, TDTR uses a trial length of $T = 20$ steps (about one trading month) to ensure that a new challenger model is genuinely better before adopting it. When drift is detected, the system retrains on the most recent $R = 200$ observations (roughly one trading year) to learn the new market regime. To maintain stability and prevent over-reacting to noise, a 200-step cooldown is enforced between updates. Finally, performance is averaged across 30 independent synthetic datasets per drift type and 30 random S&P 500 starting seeds, using identical data splits as RQ1.

H. RQ3: Setup

RQ3 builds on RQ2 by replacing the drift detector with a fixed-interval retraining schedule, serving as an ablation study that isolates the contribution of explicit drift detection. Instead of waiting for a detected drift event, the active model is retrained every 200 steps using the most recent $R = 200$ observations. All other settings, including model architectures, challenger pool size, trial length, retraining window, and data splits, remain identical to RQ2. This allows a direct comparison between detector-triggered and unconditional periodic retraining under identical computational budgets.

I. RQ4: Setup

RQ4 extends the TDTR evaluation to two conventional machine learning models: Random Forest and Support Vector Regression (SVR). Both methods are evaluated under the same TDTR configuration as RQ2 Phase 2, using the same drift detector and the same datasets. This experiment tests whether the same ceiling effect observed in real-world data also appears in non-neural models. The same train, validation, and test splits are used as in RQ2. These models were selected as they represent well-established non-neural baselines to compare with RQ2.

J. RQ5: Setup

RQ5 simulates a more realistic setting in which models are trained on a limited portion of the series and then evaluated on unseen and recurring concept configurations. This is implemented by setting the training ratio to 0.1 so that the models are trained only on the first concept. This ensures that the test set contains concept configurations that have not been encountered during initial training. All other settings remain identical to RQ2.

TABLE I: Real Data Results (S&P 500) — Static vs. Drift-Adaptive Models

Model	Static Models		Adaptive Models (-A)	
	Price MAE	Return MAE	Price MAE	Return MAE
ARIMA	522.73	0.00783	–	–
LSTM	34.70	0.00945	20.35	0.00749
ELM	**19.83**	**0.00726**	20.25	0.00739
PSO-LSTM	28.86	0.00785	**19.87**	**0.00724**
PSO-ELM	19.98	0.00732	20.48	0.00747

Note: Results are means over 30 runs with different seeds. Price MAE is computed on reconstructed prices, while Return_MAE is computed on log returns. Bold indicates the lowest value in each respective model category.

VI. Results & Discussion

In this section, the evaluation of the proposed models is presented on real-world and synthetic datasets to answer the research questions.

A. *Q1: To what extent does PSO-based optimisation of output layer weights improve the robustness of neural forecasting models (ELM and LSTM) under different types of concept drift?*

To evaluate the impact of PSO-based optimisation on forecasting robustness, performance is first examined on the real-world S&P500 dataset. The results in Table I show a clear improvement for the LSTM architecture when PSO is applied, with the PSO-LSTM reducing Price_MAE from 34.70 to 28.86. However, the same pattern does not hold for ELM, where the baseline model remains slightly better than PSO-ELM on both Price_MAE and Return_MAE. This indicates that PSO does not uniformly improve forecasting performance across architectures. This suggests that PSO-based optimisation affects the two architectures differently: it provides a clear benefit for LSTM, where the hidden representations are learned during training, and PSO can further optimise the final output layer that maps those representations to predictions, but offers limited additional value for ELM, where performance remains very similar to the already strong baseline. The table also highlights that **Return_MAE alone can be misleading** on real financial data. For example, the ARIMA baseline achieves a relatively low Return_MAE by largely predicting near-zero returns, which already produce a small absolute error because scaled log returns are numerically small. As a result, several models appear similar under Return_MAE even when their practical forecasting quality differs. In contrast, **Price_MAE** is measured in index points after reconstructing prices from predicted returns, where small return biases and occasional large misses can accumulate into much larger absolute errors. Therefore, Price_MAE provides the more practically informative comparison for real-world forecasting quality.

To assess robustness under controlled concept drift conditions, 120 synthetic datasets were analysed across four drift types. The Friedman test revealed a highly significant difference among models ($\chi^2 = 258.78, p < 0.001$). However, the Nemenyi post-hoc test, visualised in the Critical Difference (CD) diagram in Fig. 3, showed that PSO-ELM achieved the best average rank (1.79), followed by LSTM

TABLE II: Synthetic Data Results (Return_MAE) — Static vs. Drift-Adaptive Models

	Static Models					Drift-Adaptive Models			
Drift Type	**ARIMA**	**LSTM**	**ELM**	**PSO-LSTM**	**PSO-ELM**	**LSTM-A**	**PSO-LSTM-A**	**ELM-A**	**PSO-ELM-A**
Linear Abrupt	0.200 (0.117)	**0.040 (0.038)**	0.085 (0.064)	0.048 (0.006)	0.050 (0.031)	**0.037 (0.009)**	0.053 (0.033)	0.127 (0.121)	0.090 (0.056)
Linear Gradual	0.211 (0.088)	0.029 (0.005)	0.037 (0.010)	0.088 (0.006)	**0.023 (0.002)**	**0.027 (0.006)**	0.030 (0.012)	0.067 (0.054)	0.041 (0.015)
Non-linear Abrupt	0.227 (0.162)	0.083 (0.050)	0.057 (0.033)	0.034 (0.056)	**0.034 (0.005)**	0.059 (0.033)	**0.045 (0.018)**	0.082 (0.074)	0.075 (0.054)
Non-linear Gradual	0.124 (0.024)	0.022 (0.005)	0.019 (0.001)	0.019 (0.001)	**0.017 (0.001)**	0.020 (0.003)	**0.019 (0.002)**	0.020 (0.002)	0.019 (0.002)

Note: Mean and standard deviation over 30 runs. Bold indicates the lowest mean MAE per drift type within each respective model category (Static and Adaptive).

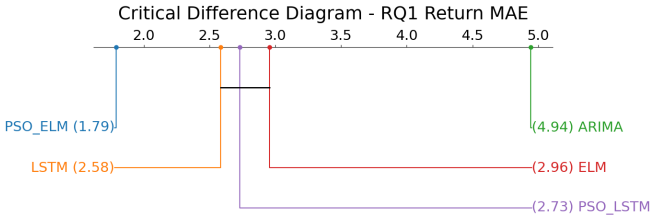

Fig. 3: Critical Difference (CD) diagram for all synthetic datasets using Return_MAE. Models connected by a horizontal bar are not significantly different according to the Nemenyi post-hoc test at $\alpha = 0.05$.

(2.58), PSO-LSTM (2.73), and ELM (2.96), while ARIMA ranked last (4.94). Notably, the horizontal bar indicates that the performance differences between LSTM, PSO-LSTM, and ELM are not statistically significant. This confirms that PSO optimisation did not provide a significant advantage over the standard LSTM baseline under mixed drift conditions on recurring concepts.

Although the aggregated ranks do not show a significant separation between the PSO variants and their baselines, performance differs across drift types. Table II shows a more differentiated robustness pattern. Under linear abrupt drift, the baseline LSTM achieves the lowest mean MAE (0.0396), outperforming both PSO variants. In contrast, under linear gradual drift, PSO-ELM performs best with a mean MAE of 0.0233, while PSO-LSTM performs substantially worse (0.0881). This suggests that PSO does not benefit both neural architectures equally and may be more useful for ELM than for LSTM in some controlled drift settings.

The benefit of PSO is most visible under the non-linear drift settings. Under non-linear abrupt drift, both PSO-based models outperform their non-PSO counterparts, with PSO-ELM achieving the lowest mean MAE (0.0338), although the margin over PSO-LSTM (0.0338) is extremely small. Under non-linear gradual drift, PSO-ELM again performs best, achieving the lowest mean MAE (0.0175). These results suggest that PSO-based optimisation is most beneficial when the drift pattern is more complex and less regular.

Overall, PSO optimisation does not provide uniform gains across all drift types. Standard models remain competitive, and in some cases, superior under simpler linear drift conditions. However, PSO-based variants, particularly PSO-ELM, show stronger robustness under more complex non-linear drift. Therefore, the main contribution of PSO in this paper is not universal superiority, but selective improvement under more challenging drift scenarios. These synthetic findings should be interpreted in the context of the benchmark design, where earlier concepts are repeated from the sixth concept onward, meaning that much of the test region reflects recurring rather than entirely novel concept configurations.

B. Q2: How effectively does drift-adaptive retraining improve forecasting performance compared to non-adaptive baselines across synthetic and real financial data?

To evaluate the effectiveness of drift-adaptive retraining, a two-phase experimental protocol was designed. *Phase 1* selected the most suitable drift detector for integration with PSO-LSTM within TDTR. *Phase 2* then evaluated the impact of adaptive retraining within TDTR across multiple forecasting models under controlled synthetic recurrent drift scenarios and real financial data.

Phase 1: Three drift detection algorithms (ADWIN, Page-Hinkley, and KSWIN) were evaluated in combination with PSO-LSTM for four synthetic drift types. The objective was to identify the detector that achieved the best drift coverage.

To maximise true drift detection, the detector sensitivity parameters were adjusted. However, increasing sensitivity improved recall at the cost of higher false positives, leading to frequent retraining events, reduced forecasting stability, and worsened MAE performance. This trade-off is visible in Table III. KSWIN detected the most true drifts (11 out of 12), but also produced the highest number of false positives ($FP = 55$). ADWIN generated fewer false positives ($FP = 23$), but missed more true drifts. Page-Hinkley performed the worst overall, detecting the fewest true drifts and still producing a substantial number of false positives.

Among the evaluated detectors, KSWIN achieved the highest drift detection coverage, particularly under non-linear and abrupt drift conditions. Its distribution-based mechanism, based on the Kolmogorov–Smirnov test, enables it to detect broader changes in the underlying data distribution rather than relying solely on shifts in central tendency. Consequently, KSWIN was selected for the *Phase 2* TDTR experiments, with the understanding that TDTR's stabilisation mechanisms (cooldown and trial-based promotion) would help reduce the negative impact of false positive triggers.

TABLE III: Drift detection performance across four synthetic drift types.

Detector	False Positive (FP)	True Drifts	Detected	Recall
Page-Hinkley	33	12	7	0.58
ADWIN	23	12	9	0.75
KSWIN	55	12	11	**0.89**

Note: Each detector was evaluated on 12 known drift points (3 per drift type).

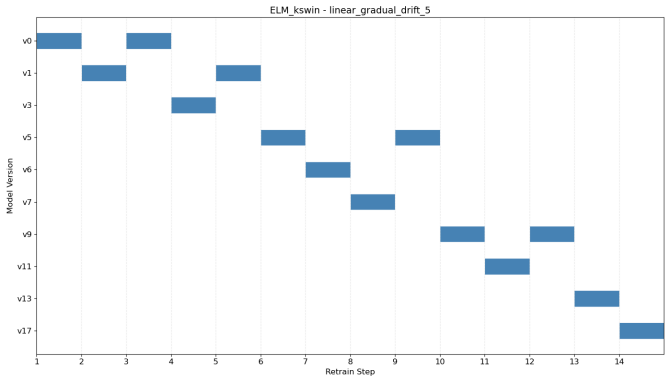

Fig. 4: TDTR model pool activity over time. The active model label changes after drift-triggered trials

In contrast, Page-Hinkley and ADWIN were less robust across the full range of drift settings. Page-Hinkley struggled especially under gradual drift, while ADWIN achieved better overall performance but still detected fewer true drifts than KSWIN.

Phase 2: Using the selected detector, adaptive retraining was evaluated across multiple forecasting models under four different drift types, each comprising 30 independent synthetic time series to estimate performance variability. In an initial detector-triggered retraining setup without trial-based selection (i.e., no challenger pool), a high detector sensitivity caused frequent false-positive retraining events. This led to excessive model updates and degraded forecasting stability resulting in roughly 20% worse aggregate MAE compared to the TDTR configuration.

To mitigate this issue, two stabilisation strategies were introduced. First, a cooldown mechanism was implemented. After each retraining event, drift signals that occurred within a predefined time interval were ignored, reducing rapid consecutive retraining triggered by noise. Second, a concurrent validation strategy was adapted: upon drift detection, a newly retrained challenger model was deployed alongside the current active model. Both models were evaluated in parallel over a fixed horizon and the model achieving a lower forecast error was retained. This strategy mitigates false-triggered model switching and improves model stability.

In the initial TDTR configuration, drift-triggered retraining produces a single challenger that competes directly against the active model over the trial horizon. If the challenger performs worse, it is permanently discarded. However, Fig. 4 suggests that retaining previously replaced models in the challenger pool can be beneficial: a model that was outperformed in one regime may later be promoted again if the data distribution reverts or recurring drift occurs, providing a lightweight form of model memory across regimes.

TABLE IV: Adaptive Behaviour on Real-World Data (S&P500)

Model	Number Retrains	Total Retrain Time (s)	Avg Retrain Time (s)
LSTM-A	**7.70**	1.11	0.14
ELM-A	7.55	**0.005**	**0.001**
PSO-LSTM-A	8.53	99.07	11.61
PSO-ELM-A	7.20	48.36	6.72

Note: Values are means over 30 runs (different seeds). Total retrain time measures cumulative update time across the test stream. Avg retrain time is the mean time per retraining event.

Table I summarises the results on the S&P 500 dataset. The clearest result is that drift-adaptive retraining helps the LSTM-based models a lot. The static LSTM has a Price_MAE of 34.70, while LSTM-A reduces this to 20.35 (approximately a 41% reduction). This suggests that when market conditions change, the offline LSTM begins to accumulate systematic errors, and retraining with recent data helps correct them. PSO-LSTM shows the same trend, improving from 28.86 to 19.87 (about a 31% reduction), confirming that TDTR is effective for gradient-based sequence models in non-stationary settings. Among the adaptive variants, PSO-LSTM-A achieves the best performance, with the lowest Price_MAE (19.87) and the return_MAE (0.00724).

In contrast, ELM-based models do not benefit much from drift-triggered adaptation on this dataset. The static ELM already achieves a low Price_MAE of 19.83, while ELM-A slightly worsens to 20.25. PSO-ELM shows a similar pattern, moving from 19.98 to 20.49. This indicates that ELM already generalises well on S&P 500. Retraining on a short recent window can replace this stable solution with a more local fit that is sensitive to noise, increasing variance without reducing bias. Since financial returns have a low signal-to-noise ratio, meaning the predictable structure is weak relative to random fluctuations, performance can quickly reach an error floor where additional retraining mainly increases variance rather than improving generalisation.

In terms of adaptation behaviour, Table IV shows that the detector triggers a similar number of retraining events across models (approximately 7-9 retrains on average). This indicates that KSWIN fires at a comparable rate regardless of architecture and that performance differences mainly come from how each model adapts to retraining rather than how often drift is detected. However, the key difference is the cost of retraining. PSO-LSTM-A and PSO-ELM-A are much more expensive to update because PSO runs an optimisation loop at each retrain (99.07s and 48.36s total retraining time, or 11.61s and 6.72s per retrain). In comparison, LSTM-A retraining is relatively inexpensive (1.11s total, about 0.14s per retrain), and ELM-A updates are near-instantaneous.

Across the 120 synthetic series, the Friedman test on **Return_MAE** confirms significant performance differences between models $p = 1.8 \times 10^{-33}$. In the Nemenyi comparison (Fig. 5), **PSO-ELM** achieves the best average rank (2.50), while **ELM-A** ranks worst overall (6.36). Several mid-ranked models, including **LSTM-A**, **PSO-LSTM-A**, **LSTM**, **PSO-ELM-A**, and **PSO-LSTM**, appear in overlapping non-significant groups, indicating that there is no statistically

consistent performance difference across datasets.

The drift-type breakdown (Table II) shows that adaptation is not uniformly beneficial. Under **gradual drift**, where the distribution evolves slowly, **static PSO-ELM** performs best (Return_MAE $= 0.0233$ for linear gradual and $= 0.0175$ for non-linear gradual), while adaptive variants often remain close to their static baselines. Under **non-linear abrupt drift**, adaptation becomes more relevant, but the strongest results are still achieved by **static PSO models** (PSO-ELM and PSO-LSTM both ≈ 0.0338). In contrast, **LSTM-A** improves clearly over static LSTM (0.0586 vs 0.0829), suggesting that retraining the full network enables better recovery after sudden structural changes than updating only the FC layer. However, these advantages are limited by the recurrent nature of the benchmark, since later regimes often resemble patterns already encountered during initial training.

The results highlight a clear contrast between how recurrent and feed-forward architectures handle non-stationary data. Under synthetic non-linear abrupt drift (Table II), the static ELM (0.057) degrades less severely than static LSTM (0.083).

These differences are deeply tied to how each architecture processes streaming data. While model *weights* remain frozen until drift is detected, the LSTM maintains an internal recursive cell state updated with every observation. Because its gating mechanisms (specifically the forget gate) were optimised for the original data distribution, they fail to discard obsolete patterns as the data distribution shifts. Consequently, this memory accumulates errors and passes corrupted representations forward, accelerating misalignment *even before retraining occurs*. As a result, LSTM models are highly responsive to adaptive retraining (LSTM-A), which successfully resets and recalibrates the internal state and realigns the weights. In contrast, ELM is a strictly feed-forward architecture with no recursive memory, relying entirely on a fixed sliding window. Thus, ELM-based models capture a highly stable global mapping during initial training without accumulating state-driven errors, allowing the static model to naturally resist drift. However, under minor or gradual drift, forcing the ELM to retrain on a short recent window (ELM-A) replaces its well-generalised global fit with a noise-sensitive local fit. This explains why adaptive ELM variants can underperform their robust static counterparts, whereas LSTMs strictly require adaptation to survive.

The behaviour of the KSWIN detector is summarised in Appendix A Table V. Across drift types, the detector triggers a broadly similar number of retraining events regardless of the forecasting architecture, indicating that detection behaviour is largely independent of the model used. Abrupt drift scenarios are detected more reliably, with around 2.8 to 3 true detections on average out of three true drift points. In contrast, gradual drift scenarios produce fewer true positives, particularly under **non-linear gradual drift**, where all adaptive models show their lowest detection coverage except **LSTM-A**, which retains slightly better sensitivity. This pattern appears to be specific to the non-linear gradual recurring setting, where changes are introduced slowly while still remaining partly similar to earlier concepts. As a result, the error signal stays relatively weak and spreads over time, making it harder to separate true drift from the normal fluctuations in the data, and therefore reducing true positive detection.

The table also highlights substantial differences in retraining cost, which is statistically confirmed by the CD diagram for total retraining time (Appendix A, Fig. 6). The Nemenyi test reveals that all four adaptive models differ significantly in speed with no overlapping groups. ELM-A ranks fastest, followed by LSTM-A. In contrast, the PSO-based variants (PSO-ELM-A and PSO-LSTM-A) rank significantly worse. Each update requires an additional swarm optimisation loop, resulting in a much higher cumulative retraining time.

Overall, RQ2 shows that TDTR is not universally beneficial but is most effective when the underlying model can lose alignment with changing regimes. LSTM-based models benefit more consistently from adaptive retraining, likely because recurrent networks can gradually forget previously useful patterns as their internal state evolves. In contrast, strong static baselines such as PSO-ELM remain highly competitive on synthetic recurring-drift benchmark, suggesting that when later concepts largely repeat patterns already observed during initial training, a well-fitted analytical model may already capture the relevant structure and therefore require little or no adaptive updating. This suggests that adaptive updating may be redundant in recurring environments where no fundamentally new patterns are introduced. The results also reveal a clear computational trade-off, since PSO-based adaptive models introduce substantially higher retraining overhead. RQ4 and RQ5 extend this analysis by evaluating simpler machine learning models and by testing whether adaptive methods become more beneficial under settings that include more genuinely unseen concepts.

During synthetic experiments, a small number of ELM runs produced anomalously large errors, concentrated in specific seeds on the non-linear drift series. In extreme cases, the Return_MAE exceeded 600, when using the standard ELM closed form solution based on the Moore-Penrose pseudoinverse $\boldsymbol{\beta} = \mathbf{H}^{\dagger}\mathbf{y}$. This behaviour is consistent with numerical instability when the hidden-layer design matrix $\mathbf{H}$ is ill-conditioned or near-singular, causing the pseudoinverse solution to amplify noise and produce extreme output weights. To mitigate this, the pseudoinverse solve was replaced with an L2-regularised least-squares (ridge regression), with $\lambda = 10^{-4}$, which improves conditioning prior to inversion and substantially reduces these catastrophic outliers [23].

$$\boldsymbol{\beta} = (\mathbf{H}^T\mathbf{H} + \lambda\mathbf{I})^{-1}\mathbf{H}^T\mathbf{y} \tag{1}$$

C. Q3: How does continuous online retraining without a drift detector compare to TDTR under identical retraining windows and computational constraints across different drift types?

RQ3 evaluates whether explicit drift detection is necessary for effective adaptation by replacing KSWIN with fixed-interval retraining every 200 steps, the same cooldown step used in TDTR, applied to the same recurring concept benchmarks.

The results show that continuous retraining achieves broadly similar forecasting accuracy (Return_MAE) to TDTR. On the real-world S&P 500 data, the no detector variants produce Price_MAE and Return_MAE values within a narrow margin of the adaptive models in Table I, with no consistent winner

across architectures. Table VI and Table VII show a similar pattern across synthetic drift types, with no detector model performing comparably or occasionally slightly better, particularly under non-linear gradual drift where both strategies converge to Return_MAE values in the range of $0.019-0.020$. This convergence is expected given the recurring structure of the benchmark, where later concepts largely repeat patterns already observed during training, reducing the advantage of any retraining strategy over another.

However, this equivalent forecasting accuracy comes at a substantial computational cost. On the real-world data, TDTR triggers between 7.2 and 8.5 retrains on average depending on the model, while fixed-interval produces approximately double the number of updates, around 17 events over the same test horizon. On synthetic data the disparity is even larger, with fixed-interval retraining reaching 39 updates (consistent with 4 concept test region spanning 8,000 steps at a 200 step interval), compared to the 6 to 15 events triggered by KSWIN (4x more).

These findings show that while a fixed retraining schedule can match TDTR in accuracy, it is highly inefficient because it forces unnecessary model updates even when the market is stable. It is also worth noting that the 200 step interval was chosen to match the TDTR retraining window rather than to optimise the fixed schedule itself. While a longer interval minimises unnecessary updates, a shorter one improves responsiveness to gradual drift. However, neither can be optimised without prior knowledge of the drift regime, which is precisely what a drift detector provides. Ultimately, relying on a drift detector proves to be much more efficient. The ablation study clearly shows that it is better to wait for an actual drift trigger than to simply retrain on a blind schedule.

D. Q4: Can conventional machine learning methods such as Random Forest and Support Vector Regression (SVR) benefit from drift adaptation within the TDTR framework, and do they exhibit the same ceiling effect observed in analytical neural models?

In RQ4, the evaluation is expanded to Random Forest (RF) and Support Vector Regression (SVR) to test if conventional ML models experience the same ceiling effect as the neural models tested in RQ2. On the S&P 500 dataset in Table VIII, the static RF model already performs near the theoretical error floor (Price_MAE 20.46). Because it is already at this performance ceiling, the adaptive RF-A variant actually performs slightly worse despite additional retraining. However, SVR shows a completely different result: the baseline static SVR is much weaker (Price_MAE 27.50), meaning it is well above the ceiling. Consequently, adaptation provides a clear benefit, with SVR-A improving the Price_MAE to 23.71. This confirms that the "ceiling effect" acts as an absolute performance barrier around a Price_MAE of 20 for this dataset. If a model is already at this ceiling (like RF or ELM), TDTR cannot improve it; but if a model is above the ceiling (like SVR or LSTM), TDTR successfully drives the error down towards it.

On synthetic data, adaptation produces mixed results depending on the drift type. Table IX shows that under gradual and recurring drift, both models worsen slightly after retraining, with RF increasing from 0.028 to 0.033 under linear gradual and SVR from 0.025 to 0.032 under non-linear gradual, consistent with the ELM finding in RQ2. Under non-linear abrupt drift, however, both models improve substantially, with RF-A reducing MAE from 0.073 to 0.044 and SVR-A from 0.112 to 0.070. This proves that weaker static baselines have much more room to recover under sudden shifts. Ultimately, it confirms that the ceiling effect primarily restricts models under gradual or recurring drift, where the baseline is already strong enough.

E. Q5: Does the TDTR framework improve forecasting performance over non-adaptive baselines when models are trained on limited concept coverage and evaluated on unseen concept configurations?

Before presenting the main results, Figures 7 and 8 (Appendix D) illustrate exactly how static models handle new versus repeated data. When tested on linear abrupt drift after training on only 30% of the data, both models show a clear "spike and recovery" pattern. Errors rise when the models encounter genuinely unseen concepts (red shading), but drop sharply as soon as older, familiar patterns reappear (green shading), all without any retraining. This shows that performance drops are driven strictly by the change in data, not simply by the passage of time. Interestingly, the error during the very first "unseen" concept (steps 0-2000) stays surprisingly low. An inspection of the underlying data generator explains why: the mathematical rules (AR coefficients) used to generate Concept 5 are nearly identical to Concept 2, which the model already learned during its initial training. Because these two concepts are mathematically so similar, the static model is able to generalise to Concept 5 without needing to adapt. However, when the model subsequently encounters Concepts 6 and 7 (structurally very different), the errors finally explode. This confirms that a well-trained static model degrades only when faced with a genuine structural shift in the data, rather than just crossing a superficial boundary into a "new" concept.

RQ5 simulates a more realistic deployment scenario by training the models on a very limited dataset (just the first concept, or 10% of the series) and then testing them on unseen configurations. This mirrors the real world, where future data can often be completely different from what was available during initial training. By doing so, it can be directly assessed whether TDTR is actually capable of adapting to genuinely new concepts that it has never seen before.

Tables X show that compared to RQ2, static model performance worsens considerably across all architectures under the limited training setting, which is expected given that the test region now contains concepts the models have never encountered during training. The performance drop is worst for ELM and LSTM, where the error roughly triples under abrupt drift compared to the RQ2 results. This clearly shows that both architectures struggle to generalise unless they have seen similar patterns during training. Against this weaker baseline, TDTR produces the most consistent improvements observed throughout the study. The gains are most significant for LSTM and ELM under linear abrupt drift, where LSTM-A reduces Return_MAE from 0.137 to 0.046, an improvement of approximately 66%, while ELM-A reduces from 0.306 to 0.090, an improvement of 70%. Similar patterns hold under

non-linear abrupt drift, where LSTM-A improves by 58% and ELM-A by 57%. These gains are substantially larger than anything observed in RQ2, confirming that TDTR is most valuable when the static model has not encountered the incoming concept during training.

The biggest improvements under unseen concepts come from LSTM and ELM, simply because their static baselines are the weakest. Retraining on a rolling 200 step window provides enough of a local signal to realign them with the new data, producing huge gains. Interestingly, a fully retrained LSTM-A actually beats the partially-updated PSO-LSTM-A under linear abrupt (0.046 vs 0.086) and linear gradual drift (0.041 vs 0.050). This proves that when encountering a truly new concept, updating only the output layer is insufficient. However, the exact opposite pattern holds for ELM. PSO-ELM-A consistently beats standard ELM-A across all drift types, reaching an impressive 49% improvement under non-linear abrupt drift (0.043 vs 0.085). Because PSO re-optimises the input weights at every retrain, the model can actively reshape exactly how features are projected to fit the new distribution before applying the analytical solve, making it far more adaptive than plain ELM-A.

However, PSO-ELM and PSO-LSTM show much smaller and less consistent gains from adaptation. Under abrupt drift where changes are sudden and concepts are genuinely new, PSO-ELM-A does safely improve over its static counterpart (0.061 vs 0.074 linear, 0.043 vs 0.049 non-linear). This confirms that adaptation still helps to maintain strong baselines when they reach truly new data. Conversely, PSO-LSTM-A makes a very weak case for adaptation, actually performing worse than its static version under linear abrupt drift (0.086 vs 0.076). This perfectly demonstrates the danger of freezing the recurrent layer when faced with completely unseen patterns. Finally, under gradual drift, the static PSO models remain highly competitive. Because gradual transitions produce perfectly slow, weak error signals, KSWIN detects them less reliably, which limits retraining events and prevents adaptation from providing any meaningful benefit.

Overall, RQ5 confirms that TDTR is most effective when faced with genuinely unseen concepts after limited training. This directly addresses the limitation found in RQ2, where recurring concepts cancel out the benefits of adaptation. Ultimately, the value of adaptation depends entirely on the quality of the static baseline: models that generalise poorly gain massive benefits from drift-triggered retraining. Conversely, PSO-based models remain highly competitive without adaptation even under unseen concepts, suggesting that swarm-based optimisation provides an implicit robustness that drastically reduces the need for online updating. These findings show that TDTR is not a universal necessity. Instead, it is a powerful, highly targeted solution that shines brightest when initial training data is simply insufficient to handle the reality of live deployment.

VII. Conclusion

This paper proposed and evaluated the Trial-Based Drift-Triggered Retraining (TDTR) framework, a highly adaptive forecasting system that combines KSWIN concept drift detection with both neural and conventional machine learning models. Across comprehensive synthetic benchmarks and real-world S&P 500 data, the results demonstrated that while swarm-based optimisation (PSO) creates incredibly strong static baselines, financial forecasting is ultimately restricted by an absolute performance ceiling. Consequently, TDTR delivers its greatest value not on models that already generalise exceptionally well, but when weaker baselines are violently challenged by genuinely unseen, abrupt structural shifts. Ultimately, by proving that fixed-interval retraining wastes immense computational resources compared to targeted adaptation, this research demonstrates the potential of TDTR in complex environments where concepts change frequently. Although ensemble-based methods that maintain multiple active models can offer broader drift coverage, they come at a much higher computational cost. TDTR deliberately keeps only one active model at a time, and the results show that this is enough to consistently outperform static offline baselines while only retraining when drift is actually detected, making it a more practical choice when computational resources are limited.

Future Work: Future research should address the architectural limitations identified within the TDTR framework to improve adaptability under strict online latency constraints. Key areas for future work include: (i) exploring dynamic partial retraining strategies, such as safely unfreezing the recurrent and output layers simultaneously, to improve LSTM responsiveness when encountering entirely new concepts; (ii) extending the framework to multivariate financial forecasting and incorporating multidimensional drift detection mechanisms to filter out false positives triggered by irrelevant market noise; (iii) replacing the fixed 200-step retraining window with a dynamic, self-tuning interval to provide a better balance between computational efficiency and accuracy across varying drift speeds; and (iv) detection ensembles, replacing the single KSWIN detector with an ensemble of complementary algorithms to improve detection coverage.

Data availability: The source code and datasets used in this paper are available at: https://git.cs.bham.ac.uk/projects-2025-26/wlb370

Acknowledgment

The authors would like to thank Dr. Leandro L. Minku for his mentorship, insightful suggestions during the course of the study, and continuous support throughout the development of this research.

REFERENCES

[1] A. Azeem, I. Ismail, S. S. Mohani, K. U. Danyaro, U. Hussain, S. Shabbir, and R. Z. Bin Jusoh, "Mitigating concept drift challenges in evolving smart grids: An adaptive ensemble lstm for enhanced load forecasting," *Energy Reports*, vol. 13, pp. 1369–1383, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2352484724008928

[2] S. Kaushik, A. Choudhury, P. K. Sheron, N. Dasgupta, S. Natarajan, L. A. Pickett, and V. Dutt, "Ai in healthcare: time-series forecasting using statistical, neural, and ensemble architectures," *Frontiers in big data*, vol. 3, p. 475663, 2020.

[3] A. Moghar and M. Hamiche, "Stock market prediction using lstm recurrent neural network," *Procedia Computer Science*, vol. 170, pp. 1168–1173, 2020, the 11th International Conference on Ambient Systems, Networks and Technologies (ANT) / The 3rd International Conference on Emerging Data and Industry 4.0 (EDI40) / Affiliated Workshops. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877050920304865

[4] X. Li, P. Wu, and W. Wang, "Incorporating stock prices and news sentiments for stock market prediction: A case of hong kong," *Information Processing & Management*, vol. 57, no. 5, p. 102212, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0306457319307952

[5] A. Tsymbal, "The problem of concept drift: Definitions and related work," 05 2004.

[6] J. a. Gama, I. Žliobaitundefined, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Comput. Surv.*, vol. 46, no. 4, Mar. 2014. [Online]. Available: https://doi.org/10.1145/2523813

[7] F. Bayram, B. S. Ahmed, and A. Kassler, "From concept drift to model degradation: An overview on performance-aware drift detectors," *Know.-Based Syst.*, vol. 245, no. C, Jun. 2022. [Online]. Available: https://doi.org/10.1016/j.knosys.2022.108632

[8] R. Pears, S. Sakthithasan, and Y. S. Koh, "Detecting concept change in dynamic data streams: A sequential approach based on reservoir sampling," *Machine learning*, vol. 97, no. 3, pp. 259–293, 2014.

[9] P. M. Gonçalves, S. G. de Carvalho Santos, R. S. Barros, and D. C. Vieira, "A comparative study on concept drift detectors," *Expert Systems with Applications*, vol. 41, no. 18, pp. 8144–8156, 2014. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0957417414004175

[10] D. M. Q. Nelson, A. C. M. Pereira, and R. A. de Oliveira, "Stock market's price movement prediction with lstm neural networks," in *2017 International Joint Conference on Neural Networks (IJCNN)*, 2017, pp. 1419–1426.

[11] T. Fischer and C. Krauss, "Deep learning with long short-term memory networks for financial market predictions," *European Journal of Operational Research*, vol. 270, no. 2, pp. 654–669, 2018. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0377221717310652

[12] W. Bao, J. Yue, and Y. Rao, "A deep learning framework for financial time series using stacked autoencoders and long-short term memory," *PloS one*, vol. 12, no. 7, p. e0180944, 2017.

[13] Y.-F. Zhang, Q. Wen, X. Wang, W. Chen, L. Sun, Z. Zhang, L. Wang, R. Jin, and T. Tan, "Onenet: Enhancing time series forecasting models under concept drift by online ensembling," 2023. [Online]. Available: https://arxiv.org/abs/2309.12659

[14] G. H. F. M. Oliveira, G. G. Cabral, A. C. F. M. Oliveira, M. G. F. M. Oliveira, L. L. Minku, and A. L. I. Oliveira, "Dynamic swarm intelligence for time series forecasting in the presence of concept drift," *SN Comput. Sci.*, vol. 6, no. 6, Jul. 2025. [Online]. Available: https://doi.org/10.1007/s42979-025-04247-z

[15] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of ICNN'95 - International Conference on Neural Networks*, vol. 4, 1995, pp. 1942–1948 vol.4.

[16] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, p. 1735–1780, Nov. 1997. [Online]. Available: https://doi.org/10.1162/neco.1997.9.8.1735

[17] M. Clerc and J. Kennedy, "The particle swarm - explosion, stability, and convergence in a multidimensional complex space," *Trans. Evol. Comp*, vol. 6, no. 1, p. 58–73, Feb. 2002. [Online]. Available: https://doi.org/10.1109/4235.985692

[18] G. Kumar, U. P. Singh, and S. Jain, "An adaptive particle swarm optimization-based hybrid long short-term memory model for stock price time series forecasting," *Soft Computing*, vol. 26, no. 22, pp. 12 115–12 135, 2022.

[19] A. Bifet and R. Gavalda, "Learning from time-changing data with adaptive windowing," in *Proceedings of the 2007 SIAM international conference on data mining*. SIAM, 2007, pp. 443–448.

[20] C. Raab, M. Heusinger, and F.-M. Schleif, "Reactive soft prototype computing for concept drift streams," *Neurocomputing*, vol. 416, pp. 340–351, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0925231220305063

[21] D. V. Hinkley, "Inference about the change-point from cumulative sum tests," *Biometrika*, vol. 58, no. 3, pp. 509–523, 1971.

[22] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine learning research*, vol. 7, no. Jan, pp. 1–30, 2006.

[23] R. Cancelliere, M. Gai, P. Gallinari, and L. Rubini, "Ocrep: an optimally conditioned regularization for pseudoinversion based neural training," *Neural Networks*, vol. 71, pp. 76–87, 2015.

APPENDIX A
EXTENDED RESULTS: DRIFT-ADAPTIVE RETRAINING (RQ2)

This appendix provides supplementary data on detection behaviour and critical difference diagrams for the adaptive models evaluated in RQ2.

TABLE V: Detection & retraining behaviour across synthetic drift scenarios (30 runs mean).

Drift Type	Model	Drifts Detected	True Positives	Total Retrain Time (s)
Linear Abrupt	ELM-A	17.37	2.90	0.02
	LSTM-A	14.73	2.80	2.19
	PSO-ELM-A	15.47	2.97	52.60
	PSO-LSTM-A	15.07	2.83	176.36
Linear Gradual	ELM-A	15.30	2.57	0.01
	LSTM-A	12.83	2.27	2.06
	PSO-ELM-A	14.10	2.67	61.86
	PSO-LSTM-A	12.63	2.20	145.51
Non-linear Abrupt	ELM-A	16.13	2.83	0.01
	LSTM-A	17.47	2.93	2.38
	PSO-ELM-A	14.63	2.83	37.23
	PSO-LSTM-A	14.93	2.87	129.24
Non-linear Gradual	ELM-A	8.50	1.63	0.01
	LSTM-A	10.23	2.13	1.34
	PSO-ELM-A	6.93	1.67	34.79
	PSO-LSTM-A	6.97	1.43	107.67

Note: Mean values over 30 runs per drift type. -A denotes drift-adaptive variant using KSWIN with trial-based model selection.

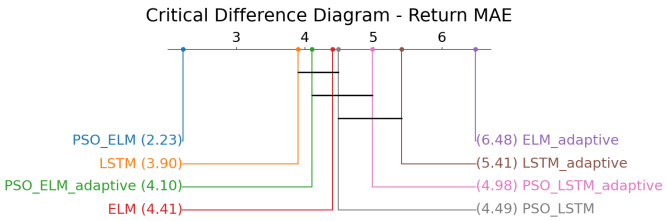

Fig. 5: Critical Difference (CD) diagram for all synthetic datasets using Return_MAE. Models connected by a horizontal bar are not significantly different according to the Nemenyi post-hoc test at $\alpha = 0.05$. A lower average rank (further left) indicates better forecasting performance.

APPENDIX B
EXTENDED RESULTS: CONTINUOUS RETRAINING COMPARISON (RQ3)

This appendix outlines the continuous-update algorithm and presents the corresponding performance tables without explicit

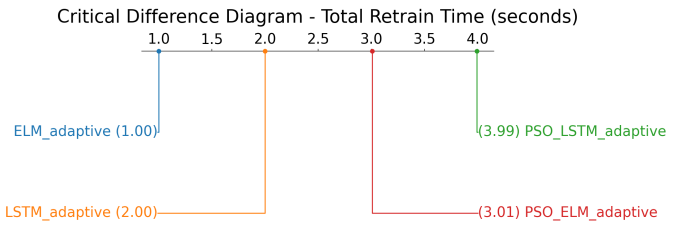

Fig. 6: Critical Difference (CD) diagram for all synthetic datasets using Total Retrain Time. A lower average rank (further left) indicates better forecasting performance.

drift detection.

Algorithm 2 Continuous-Update Variant of TDTR without Drift Detection

Train initial active model M_{active} on historical data
Initialise challenger pool $\mathcal{P} \leftarrow \emptyset$, buffer $B \leftarrow [\,]$, cooldown $\leftarrow 0$, update interval K
for each t from w to $|X_{\text{test}}| - 1$ **do**
  Predict $\hat{y}_t$ with M_{active}, compute $e_t = |\hat{y}_t - y_t|$, append y_t to B
  Update rolling errors for M_{active} and all $M \in \mathcal{P}$
  if trial is active and trial steps $= T$ **then**
    Promote the model in $\{M_{\text{active}}\} \cup \mathcal{P}$ with lowest trial MAE
    Reset error histories, set cooldown $\leftarrow C$, end trial
  end if
  if cooldown > 0 **then**
    cooldown $\leftarrow$ cooldown -1; **continue**
  end if
  if $t \bmod K = 0$ and $|B| \geq R$ **then**
    $M' \leftarrow \textsc{Retrain}(\textsc{DeepCopy}(M_{\text{active}}), B[-R:])$
    Insert M' into $\mathcal{P}$, evict worst if $|\mathcal{P}| > P_{\max}$, start trial
  end if
end for

TABLE VI: Real Data Results without Drift Detection (S&P500)

Model	Price MAE	Return_MAE
PSO-LSTM	20.0229	**0.00728**
LSTM	**19.9405**	0.00731
ELM	20.4342	0.00743
PSO-ELM	20.6874	0.00752

Note: Results are presented as the mean value across all runs. Values in bold represent the lowest MAE.

TABLE VII: Return_MAE across synthetic drift scenarios (no detector, mean values over 30 runs).

Drift Type	LSTM	PSO-LSTM	PSO-ELM	ELM
Linear Abrupt	**0.035 (0.009)**	0.055 (0.031)	0.060 (0.041)	0.095 (0.075)
Linear Gradual	**0.027 (0.005)**	0.034 (0.011)	0.038 (0.014)	0.049 (0.022)
Non-linear Abrupt	0.051 (0.029)	**0.051 (0.031)**	0.053 (0.040)	0.069 (0.056)
Non-linear Gradual	**0.019 (0.004)**	0.020 (0.003)	0.019 (0.002)	0.020 (0.001)

Note: Mean (std) over 30 runs. Models retrain at fixed intervals with no drift detector. Bold indicates lowest mean MAE per drift type.

APPENDIX C
EXTENDED RESULTS: CONVENTIONAL MACHINE LEARNING MODELS (RQ4)

This appendix provides the full performance breakdown for the Random Forest and SVR models evaluated in RQ4.

TABLE VIII: Real Data Results (S&P 500) — Static vs. Adaptive ML Models (RQ4)

Model	Static Models		Adaptive Models (-A)	
	Price MAE	Return MAE	Price MAE	Return MAE
RF	**20.46**	**0.00749**	20.65	0.00756
SVR	27.50	0.01005	**23.71**	**0.00913**

Note: Mean values over 30 runs. -A denotes drift-adaptive variant using KSWIN. Bold indicates lowest MAE between static and adaptive for each model.

TABLE IX: Synthetic Data Results (Return_MAE) — Static vs. Adaptive ML Models (RQ4)

Drift Type	Static Models		Adaptive Models (-A)	
	RF	SVR	RF-A	SVR-A
Linear Abrupt	**0.069 (0.064)**	0.139 (0.136)	**0.072 (0.033)**	0.106 (0.058)
Linear Gradual	**0.028 (0.012)**	0.042 (0.033)	**0.033 (0.011)**	0.043 (0.020)
Non-linear Abrupt	**0.073 (0.215)**	0.112 (0.296)	**0.044 (0.042)**	0.070 (0.059)
Non-linear Gradual	**0.017 (0.001)**	0.025 (0.003)	**0.019 (0.002)**	0.032 (0.007)

Note: Mean (std) over 30 runs. -A denotes drift-adaptive variant using KSWIN. Bold indicates lowest MAE per drift type within each respective model category.

APPENDIX D
EXTENDED RESULTS: EVALUATION ON UNSEEN CONCEPTS (RQ5)

This appendix details the experimental results evaluating model performance when tested on unobserved concept configurations.

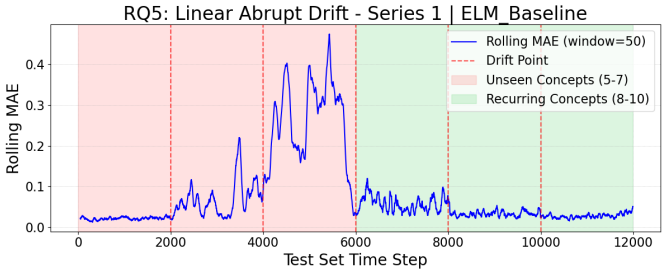

Fig. 7: Rolling MAE of static ELM baseline on linear abrupt drift (Series 1) under a diagnostic split (train_ratio=0.3, val_ratio=0.1). Error rises during unseen concepts (5–7, red region) and drops sharply when recurring concepts (8–10, green region) are encountered, confirming the palindromic concept structure.

TABLE X: Extended Synthetic Data Results (Return_MAE) — Static vs. Adaptive Models (RQ5)

A: Static Models				
Drift Type	**LSTM**	**PSO-LSTM**	**ELM**	**PSO-ELM**
Linear Abrupt	0.137 (0.163)	0.076 (0.060)	0.306 (0.652)	**0.074 (0.074)**
Linear Gradual	0.069 (0.064)	0.059 (0.012)	0.263 (0.744)	**0.034 (0.027)**
Non-linear Abrupt	0.156 (0.165)	0.052 (0.051)	0.198 (0.432)	**0.049 (0.091)**
Non-linear Gradual	0.030 (0.009)	0.018 (0.000)	0.024 (0.004)	**0.018 (0.001)**
B: Adaptive Models (-A)				
Drift Type	**LSTM-A**	**PSO-LSTM-A**	**ELM-A**	**PSO-ELM-A**
Linear Abrupt	**0.046 (0.005)**	0.086 (0.057)	0.090 (0.047)	0.061 (0.022)
Linear Gradual	**0.041 (0.003)**	0.050 (0.015)	0.052 (0.016)	0.039 (0.011)
Non-linear Abrupt	0.066 (0.018)	0.059 (0.060)	0.085 (0.037)	**0.043 (0.036)**
Non-linear Gradual	0.023 (0.003)	**0.019 (0.001)**	0.020 (0.001)	0.019 (0.001)

Note: Mean (std) over 30 runs. -A denotes drift-adaptive variant using KSWIN. Bold indicates lowest MAE per drift type within each section.

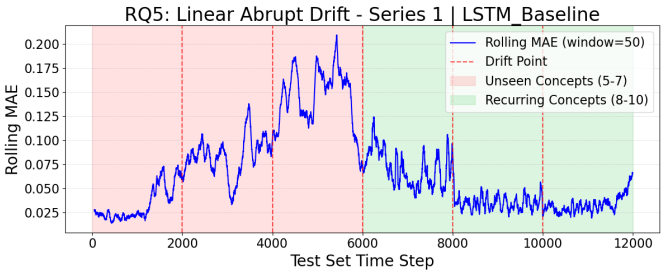

Fig. 8: Rolling MAE of static LSTM baseline on linear abrupt drift (Series 1) under the same diagnostic split as Figure 7. The same pattern is observed, with error accumulating across unseen concepts and recovering upon encountering recurring concepts.